

MONEY MASTERY

Township Economy Game

Technical Architecture Document

Version: 1.0 (Option B - Full System)

Date: February 2026

Author: NovaLearning Engineering

Target: Series A Investor Ready

Platform: Galaxy A03 Optimized

CONFIDENTIAL - NovaLearning (Pty) Ltd

Table of Contents

Note: Update this table of contents after opening in Word (Right-click > Update Field).

1. Executive Summary

Money Mastery is the world's first systemic financial literacy game designed for Grade R learners (ages 4-6), optimized for low-end Android devices prevalent in South African townships. Unlike competitors that stop at coin counting, Money Mastery teaches asset ownership, community banking (stokvels), inflation awareness, civic taxation, and debt trap recognition through an immersive 3D township economy.

Architecture Philosophy: Every technical decision prioritizes three constraints: Galaxy A03 performance (30fps minimum), POPIA-compliant privacy, and offline-first capability. The system uses a progressive complexity model where financial concepts unlock week-over-week, matching Bloom's Taxonomy progression from remembering coin values to creating personal earning strategies.

1.1 System Overview

The architecture follows a layered approach designed for progressive enhancement and offline resilience:

Layer	Technology	Purpose
Presentation	React 18 + Three.js (R128)	3D township hub, mini-games, UI overlays
State Management	Zustand + IndexedDB	Game state, economy simulation, offline persistence
Economy Engine	Custom TypeScript module	Asset valuation, inflation calc, stokvel rotation
Content Delivery	Workbook QR + Service Worker	Progressive unlock, offline caching
Analytics	Custom event logger	POPIA-compliant learning metrics
Backend (Optional)	Supabase Edge Functions	Sync, leaderboard-free community progress

2. Core Architecture

2.1 Application Flow

The core loop integrates physical workbook pages with digital gameplay through QR code triggers:

1. Student completes workbook activity page (physical)
2. Parent/teacher scans QR code on page corner
3. App launches corresponding digital township scene
4. Student plays earning mini-games (EARN phase)
5. Student makes financial decisions in shop/stokvel (DECIDE phase)
6. Township visually evolves based on decisions (SEE IMPACT phase)
7. Completion unlocks next workbook page

2.2 Module Architecture

The application is decomposed into six core modules, each independently testable and lazy-loadable:

2.2.1 Township Hub Module

Responsibility: 3D isometric township rendering, navigation, and visual state reflection.

Component	Specification	Performance Budget
Scene Graph	Three.js Scene with OrbitControls (limited)	<8K polygons total
Buildings	GLB models, baked lighting, LOD system	<500 tris each, <50KB GLB
Characters	5 max on screen, sprite-based at distance	<800 tris near, 2D sprite far
Ground Plane	Single textured quad with decals	512x512 texture atlas
Skybox	CSS gradient background (no cube map)	0 GPU cost
UI Overlay	React DOM overlay on Three.js canvas	Standard React rendering

The township hub uses an Animal Crossing-inspired isometric camera locked at 45 degrees. Building placement follows a grid system where community investments (school, clinic) become visible as players progress through tax contributions.

2.2.2 Economy Engine Module

Responsibility: All financial simulation including asset valuation, inflation modeling, stokvel rotation, and tax pooling.

The economy engine runs entirely client-side as a deterministic state machine. This ensures offline functionality and prevents server-dependent gameplay.

System	Formula/Logic	Age-Appropriate Simplification
Asset Income	$\text{daily_income} = \text{base_rate} * \text{asset_level}$	Garden earns R5/day, shown as growing coins
Inflation	$\text{price}[\text{week}] = \text{base} * (1 + 0.05 * \text{week})$	5% weekly; ice cream goes from R5 to R6
Stokvel Pool	$\text{pool} += 4 * \text{weekly_contribution}$	Visual jar fills up, rotates to each member
Tax Collection	$\text{tax} = \text{floor}(\text{income} * 0.1)$	R1 per R10 earned, shown as community coins
Debt Interest	$\text{debt} = \text{principal} * 1.4$	Mashonisa charges R70 for R50 loan, red warning
Asset Appreciation	$\text{value}[\text{week}] = \text{purchase} * (1 + 0.08 * \text{week})$	Assets grow faster than inflation line

2.2.3 Mini-Game Engine

Responsibility: Individual earning games with progressive difficulty scaling.

Tier	Game	Mechanic	Earning	Unlock Condition
1 - Labor	Coin Sorting	Drag coins to correct slots (60s)	R1-5	Default
1 - Labor	Garden Watering	Tap sprouting plants in sequence (45s)	R1-5	Default
1 - Labor	Lemonade Stand	Serve customers, give correct change (90s)	R1-5	Default
2 - Assets	Vegetable Garden	Auto-grows, tap to harvest daily	R5/day passive	Purchase: R20
2 - Assets	Chicken Coop	Collect eggs, sell at market	R3/day passive	Purchase: R15
2 - Assets	Craft Table	Beadwork auto-sells	R4/day passive	Purchase: R25
3 - Community	Shared Taxi	Group investment, dividend payouts	R10/day each	Stokvel: R100 group
3 - Community	Spaza Shop	Community store management	R8/day each	Stokvel: R80 group

Each mini-game follows a consistent lifecycle: intro animation (2s), gameplay (45-90s), reward animation (3s), decision prompt (save/spend/share). Games use 2D overlay mechanics on the 3D township background to minimize GPU load.

2.2.4 Stokvel System

Responsibility: Community banking simulation with 4 AI township kids + player.

The stokvel system is the Ubuntu differentiator. It teaches delayed gratification, trust economics, and collective wealth building through a rotating savings mechanism authentic to South African township culture.

Rotation Logic: Each week, one member of the 5-person circle (4 AI + player) receives the full pool. The player sees three phases: contribution (R5 from each), accumulation (jar fills visually), and payout (one member receives R25). When it's the player's turn, they can purchase a Tier 3 community asset.

AI Member Personalities: Each AI kid has a distinct financial personality that the player observes: Thandi (saver - always buys assets), Siphos (spender - buys toys, shows regret), Naledi (community - invests in shared projects), Bongani (balanced - mixes both). This creates natural comparison learning without didactic lectures.

2.2.5 Progressive Unlock System

Responsibility: Week-over-week concept drip-feeding aligned with workbook progression.

Week	Concepts Unlocked	Game Features	Bloom's Level
1	Earning, spending, coin values	3 labor mini-games, basic shop	Remember & Understand
2	Saving, assets, passive income	Garden purchase, asset income	Apply
3	Community saving, stokvels, group power	Stokvel circle, community assets	Analyze
4	Inflation, buying power erosion	Price changes, inflation visual	Analyze
5	Tax as investment, civic return	Township fund, community voting	Evaluate
6	Debt traps, predatory lending	Mashonisa NPC, warning system	Evaluate
7-8	Strategy creation, portfolio building	Full economy, open play	Create

2.2.6 Shop & Decision Engine

Responsibility: Purchase interface with color-coded asset vs consumption items.

The shop is the primary decision-making interface. Items are intentionally color-coded: GREEN border for assets (items that generate ongoing income), RED border for consumption (temporary joy, no lasting value). This visual language is introduced without explanation in Week 1 - children discover the pattern naturally through repeated play.

Item	Type	Cost	Return	Visual Cue
Vegetable Seeds	Asset (GREEN)	R20	R5/day ongoing	Growing plant animation
Chicken	Asset (GREEN)	R15	R3/day eggs	Egg-laying animation
Craft Supplies	Asset (GREEN)	R25	R4/day beadwork	Auto-crafting animation
Toy Car	Consumption (RED)	R10	Fun animation only	Fades after 1 play
Sweets	Consumption (RED)	R5	Happy face only	Disappears immediately
Sticker Book	Consumption (RED)	R8	Collection display	No income generation

3. Performance Architecture (Galaxy A03)

The Galaxy A03 (MediaTek Helio P35, 3GB RAM, Mali-G52 GPU) defines our performance ceiling. Every architectural decision passes through this constraint.

3.1 Rendering Pipeline

Constraint	Budget	Strategy
Total Scene Polygons	<8,000 triangles	LOD system: near (<5m) full mesh, far sprite swap
Draw Calls	<25 per frame	Texture atlas, instanced geometry, merged static meshes
Texture Memory	<16MB total	512x512 max, compressed ASTC/ETC2, shared atlases
JavaScript Bundle	<250KB gzipped	Code splitting, tree shaking, lazy module loading
Total Asset Size	<1MB initial	Progressive loading, Service Worker cache
Frame Rate	30fps minimum	requestAnimationFrame throttle, skip frames if needed
Canvas Resolution	720p max render	CSS upscale to device resolution

3.2 Memory Management

- Scene pooling: Township hub stays in memory, mini-games swap into a shared canvas
- Texture streaming: Only active scene textures loaded, LRU eviction for inactive
- Geometry instancing: All buildings share base geometries with material variants
- Audio pooling: Max 3 concurrent audio sources, Web Audio API with gain nodes
- GC pressure: Object pooling for particles, reusable typed arrays for math

3.3 Offline Architecture

Money Mastery operates fully offline after initial download. This is critical for South African townships where connectivity is intermittent and expensive.

- Service Worker: Cache-first strategy for all assets, network-first for optional sync
- IndexedDB: Full game state persisted locally with versioned schema migrations
- QR Codes: Encoded as content identifiers, not URLs - work without internet
- Sync Queue: Optional background sync when connectivity available (future feature)

4. Data Architecture

4.1 Client State Schema

All game state lives in Zustand stores persisted to IndexedDB. The schema is designed for forward migration as new features unlock:

Store	Key Fields	Persistence
PlayerStore	wallet, earnedTotal, spentTotal, savedTotal, sharedTotal	IndexedDB
AssetStore	ownedAssets[], assetLevels{}, dailyIncome, totalAssetValue	IndexedDB
StokvelStore	currentWeek, pool, rotation[], playerContributed, payoutHistory[]	IndexedDB
EconomyStore	inflationRate, currentPrices{}, taxPool, communityVotes{}	IndexedDB
ProgressStore	currentWeek, unlockedFeatures[], completedGames[], bloomLevel	IndexedDB
TownshipStore	buildings[], communityProjects[], visualUpgrades[]	IndexedDB
AnalyticsStore	sessionEvents[], decisionLog[], errorPatterns[]	IndexedDB + Optional Sync

4.2 POPIA Compliance

Money Mastery collects zero personally identifiable information. All analytics are aggregate learning patterns stored locally:

- No names, emails, or device identifiers collected
- Analytics use session-scoped random IDs (no cross-session tracking)
- Parent consent screen before any optional data sync
- All data deletable via in-app settings (POPIA right to deletion)
- No third-party analytics SDKs (no Google Analytics, Firebase, etc.)

5. Cultural Design System

The visual language is critical to authenticity and avoiding harmful stereotypes. The township aesthetic should be colorful, hopeful, and dignified - celebrating community resilience without poverty porn.

5.1 Visual Language

Element	Style Direction	Avoid
Buildings	Colorful corrugated iron, Ubuntu murals, garden patches	Dilapidated, grey, hopeless framing
Characters	Diverse skin tones, school uniforms, bright clothing	Stereotypical dress, single skin tone
Environment	Sunny, green spaces, community gathering spots	Littered, dusty, isolated scenes
UI Elements	Rounded corners, warm colors, large touch targets	Corporate/sterile, small text
Currency	Stylized Rand coins with SA wildlife	Generic coins, dollar signs
Animations	Joyful, bouncy, celebratory	Slow, sad, punitive for mistakes

5.2 Audio Design

- Background: Gentle marimba/African instrument loops, township ambiance (birds, distant laughter)
- Success sounds: Celebratory ululation samples, hand claps, joyful exclamations
- UI feedback: Soft wooden/organic click sounds (not digital beeps)
- Voice prompts: EN/AF/ZU with warm, encouraging adult voice (not robotic TTS)
- Budget: <20KB per effect, <50KB per voice clip, MP3 + WebM dual format

5.3 Language Support

Language	Priority	Coverage	Notes
English	P0	100% - all UI, voice, text	Default language
Afrikaans	P1	100% UI text, 80% voice	Western Cape primary
isiZulu	P1	100% UI text, 80% voice	KZN + Gauteng primary
isiXhosa	P2	UI text only (post-launch)	Eastern Cape expansion
Sesotho	P2	UI text only (post-launch)	Free State expansion

6. Technology Stack

6.1 Frontend

Technology	Version	Purpose	Size Impact
React	18.x	Component UI, state management bridge	~40KB gzipped
Three.js	R128	3D township rendering	~60KB gzipped (tree-shaken)
Zustand	4.x	Global state management	~2KB gzipped
GSAP	3.x	Animations (mini-games, transitions)	~25KB gzipped
Vite	5.x	Build tool, dev server, code splitting	Build-time only
Workbox	7.x	Service Worker, offline caching	~10KB gzipped
idb-keyval	6.x	IndexedDB wrapper for state persistence	~1KB gzipped

6.2 Backend (Optional - Phase 2)

Technology	Purpose	When Needed
Supabase	Database + Auth + Edge Functions	When adding parent dashboard
Supabase Edge Functions	Sync API, analytics aggregation	When adding cross-device sync
Supabase Storage	Asset CDN for progressive downloads	When asset library exceeds 5MB
Vercel	Static hosting, CDN edge deployment	Production deployment

6.3 Development & CI/CD

Tool	Purpose
GitHub	Source control, CI/CD via Actions
Vitest	Unit testing (economy engine, state logic)
Playwright	E2E testing (game flows, offline scenarios)
Lighthouse CI	Performance regression monitoring
Chromatic	Visual regression for UI components
Sentry	Error tracking (production monitoring)

7. Security & Privacy

Concern	Mitigation
Child data protection	Zero PII collection, POPIA + COPPA alignment
QR code injection	Content IDs validated against local allowlist, no URL execution
Local storage tampering	Integrity checksums on IndexedDB state (detect cheating)
Network requests	All optional, HTTPS-only, CSP headers prevent XSS
Third-party dependencies	Audited supply chain, lockfile pinning, no ad SDKs
Parent access	PIN-protected settings, no child-accessible account creation

8. Build Phases & Milestones

8.1 Phase 1: MVP Core (Weeks 1-6)

Goal: Playable demo for investor deck - Earn/Save/Spend loop with township hub.

Week	Deliverable	Definition of Done
1-2	Township Hub scaffold	3D scene renders at 30fps on Galaxy A03, camera controls work
2-3	3 Labor mini-games	Coin Sort, Garden Water, Lemonade Stand playable with scoring
3-4	Shop system + economy engine	GREEN/RED items, purchase flow, wallet state persists
4-5	Asset income system	Garden generates passive R5/day, visual feedback working
5-6	QR unlock + offline	Service Worker caching, QR scan triggers correct scene

8.2 Phase 2: Ubuntu Economics (Weeks 7-10)

Goal: Stokvel + community features that differentiate from all competitors.

Week	Deliverable	Definition of Done
7-8	Stokvel system	4 AI kids + player, rotation works, pool visuals, payout animation
8-9	Community assets (Tier 3)	Shared Taxi + Spaza Shop unlockable via stokvel investment
9-10	Township evolution visuals	Buildings upgrade based on community investment level

8.3 Phase 3: Advanced Literacy (Weeks 11-14)

Goal: Inflation, tax, and debt features that make this globally unique.

Week	Deliverable	Definition of Done
11	Inflation system	Prices change weekly, visual comparison graph, asset outperformance
12	Tax + community voting	10% tax, vote on 3 project options, see built result

13	Debt trap awareness	Mashonisa NPC, loan offer, warning system, stokvel alternative
14	Polish + testing	Full playthrough, performance pass, cultural review

8.4 Phase 4: Launch Prep (Weeks 15-16)

- Lighthouse performance audit (target: 90+ score)
- Galaxy A03 real-device testing sprint
- Cultural sensitivity review with township community members
- Stokvel mechanic validation with real stokvel practitioners
- EN/AF/ZU translation completion and voice recording
- Investor demo preparation and pitch deck alignment

9. Competitive Moat Analysis

This architecture is specifically designed to create technical and content moats that Orboot clones and generic EdTech platforms cannot replicate:

Moat Dimension	NovaLearning Advantage	Competitor Gap
Financial Depth	Asset ownership, stokvels, inflation, tax, debt	Coin counting only
Cultural Authenticity	Real SA township economics, Ubuntu values	Generic Western economy
Device Optimization	Galaxy A03 native performance	Requires iPad/premium tablet
Offline-First	Full gameplay without internet	Cloud-dependent
Community Economics	Collaborative progress, no leaderboards	Individual competition
Physical-Digital Bridge	Workbook QR integration	App-only experience
Pedagogical Rigor	Bloom's Taxonomy progression, CAPS aligned	Entertainment-first
Privacy by Design	Zero PII, POPIA compliant	Data harvesting for ads

10. Risk Register

Risk	Likelihood	Impact	Mitigation
Galaxy A03 can't hit 30fps	Medium	High	Progressive degradation: sprite-only mode, 2D fallback
Stokvel mechanic too complex for age 5	Medium	Medium	Simplify to visual jar + celebration, skip rotation detail
Debt content too scary for young children	Low	High	Red warning only, no punishment - just show alternative path
Cultural insensitivity in township visuals	Medium	Critical	Community review panel before launch, iterative feedback
Scope creep delays MVP	High	High	Phase gates: Phase 1 is investor-ready without Phase 2-4
Inflation concept too abstract	Medium	Low	Visual ice cream price comparison only, no graphs
Offline IndexedDB storage limits	Low	Medium	Compress state, purge old analytics, <5MB total budget

11. Success Metrics

11.1 Technical KPIs

Metric	Target	Measurement
Frame Rate	30fps sustained on Galaxy A03	requestAnimationFrame delta monitoring
Load Time	<3 seconds first meaningful paint	Lighthouse CI automated checks
Bundle Size	<250KB JavaScript gzipped	Vite build analysis
Offline Reliability	100% gameplay without network	Playwright offline test suite
Crash Rate	<0.1% sessions	Sentry error tracking
Asset Load	<1MB initial, <5MB total	Build-time asset budget tracking

11.2 Learning KPIs

Metric	Target	Measurement
Asset Purchase Rate	>60% choose asset over toy by Week 3	DecisionLog analysis
Stokvel Completion	>80% complete full rotation cycle	StokvelStore progress
Concept Retention	Can identify GREEN vs RED items	In-game assessment mini-game
Session Length	8-12 minutes average	SessionEvent timestamps
Return Rate	>70% return within 48 hours	Session gap analysis
Workbook Completion	>85% complete matched workbook page	ProgressStore tracking

12. Appendices

12.1 File Structure

Recommended project structure for the Money Mastery application:

```
money-mastery/  
  src/  
    components/      # React UI components  
    scenes/          # Three.js scene modules  
      township/      # Hub scene + buildings  
      minigames/     # Individual game scenes  
    engine/          # Economy simulation  
      economy.ts     # Asset, inflation, tax logic  
      stokvel.ts     # Rotation + pool management  
      shop.ts        # Purchase + decision engine  
    stores/          # Zustand state stores  
    assets/          # GLB models, textures, audio  
    i18n/            # EN/AF/ZU translations  
    workers/         # Service Worker + sync  
    utils/           # QR parser, analytics logger  
  public/  
    models/          # 3D assets (lazy loaded)  
    audio/           # Sound effects + voice  
  tests/  
    unit/            # Vitest economy engine tests  
    e2e/             # Playwright game flow tests
```

12.2 Key Design Decisions Log

Decision	Rationale	Alternative Considered
Client-side economy engine	Offline-first, no server dependency	Server-authoritative (rejected: connectivity)
Zustand over Redux	Minimal bundle, simple API, middleware support	Redux Toolkit (rejected: 15KB overhead)
Three.js R128 over R160+	Proven Galaxy A03 compatibility, smaller build	Latest Three.js (rejected: untested on target)
No leaderboards	Ubuntu values, collaborative not competitive	Competitive ranking (rejected: values conflict)
QR content IDs not URLs	Works offline, no hijacking risk	URL-based QR (rejected: requires internet)
2D UI overlays	Performance savings, familiar mobile patterns	3D UI in scene (rejected: GPU budget)

END OF DOCUMENT

NovaLearning (Pty) Ltd - Confidential